

Функции

1. Определение - Функцията е именувана, относително самостоятелна част от програмата, която се дефинира веднъж, а може да се извиква за изпълнение много пъти
2. Дефиниция на функция:
Състои се от заглавие и тяло.
Заглавието се състои от:
 - тип на връщаната от функцията стойност;
 - име на функцията, което трябва да е валиден за езика идентификатор;
 - списък от параметри (наричат се формални параметри), заградени в кръгли скоби.

Тяло на функцията:

```
{  
    оператор 1  
    оператор 2  
    ...  
    оператор N  
}
```

Пример: Дефиниция на функция за изчисляване a (реално число, различно от 0) на степен n (n - произволно цяло число):

```
float stepen (float a, int n)  
{  
    float P=1;  
    int m,i;  
    if(n>0)m=n;else m=-n;  
    for(i=1;i<=m;i++)  
        P=P*a;  
    if(n<0)P=1/P;  
    return P;  
}
```

3. Извикване на функция за изпълнение

Функцията се извиква с името си и списък от фактически параметри, като броят и типът на фактическите параметри трябва да съответства на броя и типа на формалните параметри.

Пример: Програма за изчисляване стойността на израза $a^n + b^k$

```
int main()  
{  
    float a,b,s;  
    int n,k;  
    do{
```

```

        printf("a=");
        scanf("%f",&a);
    } while(a==0);
    printf("n=");
    scanf("%d",&n);
    do{
        printf("b=");
        scanf("%f",&b);
    } while(b==0);
    printf("k=");
    scanf("%d",&k);
    s=stepen(a,n)+stepen(b,k);
    printf("s=%f",s);
    return 0;
}

```

2 задача. Да се състави функция, която извежда на екрана триъгълна фигура от символи.

Пример (5 реда от *):

```

*
* *
* * *
* * * *
* * * * *

```

```

#include<stdio.h>
void fig(int n, char c)
{
    int i,k;
    for(i=1;i<=n;i++)
    {
        for(k=1;k<=i;k++)
            printf("%2c",c);
        printf("\n");
    }
}

```

```

int main()
{
    int n;char c;
    printf("n=");scanf("%d",&n);
    getchar();
}

```

```
c=getchar();
fig(n,c);
return 0;
}
```

Параметри на функции

В езика C параметрите се предават по стойност (наричат се параметри - стойности).

Пример: Да се състави функция за размяна на стойностите на две променливи

ако дефинираме следната функция:

```
void change(int a,int b)
{
    int c;
    c=a;
    a=b;
    b=c;
}
```

и я извикаме в main():

```
int main()
{
    int x=12, y=10;
    change(x,y);
    printf("x=%d y=%d",x,y);
    return 0;
}
```

ще установим, че при извикването на функцията change(x,y), функцията НЕ разменя стойностите на фактическите параметри x и y.

Обяснение:

Изпълнението на програмата започва от главната функция. Заделя се оперативна памет за променливите x и y.

Състояние на оперативната памет (ОП):

Преди извикване на функцията change() x=12 y=10

По време на изпълнение на `change()` `x=12 y=10`

Заделя се допълнителна памет за формалните параметри и стойностите на фактическите се копират в съответните формални параметри. Стойностите на формалните параметри `a` и `b` се разменят, но стойностите на фактическите `x` и `y` не се променят, защото функцията работи с техните копия.

`a=10 b=12`

След извикване на функцията `change()` `x=12 y=10`

Паметта, заделена за формалните параметри `a` и `b` и вътрешната променлива `c` се освобождава. Остават `x` и `y` със старите им стойности.

2 вариант - функция за размяна на стойностите на две променливи с използване на външни променливи

- Външни (глобални, нелокални) променливи - променливи, дефинирани извън тялото на която и да е функция. Област на действие на външните променливи - от точката на дефинирането им до края на файла, в който са дефинирани.
- Вътрешни (локални) променливи - променливи, дефинирани в тялото на дадена функция. Област на действие на вътрешните променливи - от точката на дефинирането им до края на функцията, в която са дефинирани.

```
int x,y; //външни променливи
void change() //функцията няма параметри
{
    int c; //вътрешна променлива за функцията change()
    c=x;
    x=y;
    y=c;
}
int main()
{
    x=12; y=10;
    change();
    printf("x=%d y=%d",x,y);
    return 0;
}
```

Извеждат се: `x=10 y=12`;

Т.е. функцията `change()` разменя стойностите на `x` и `y`.

Недостатък на функцията, използваща външни променливи - разменя стойностите само на променливите x и y. Ако искаме да разменим стойностите на друга двойка променливи - трябва да дефинираме друга функция.

3 вариант - като параметри на функцията да се предават адресите на променливите x и y.

Указатели (POINTERS)

Указателят е променлива, чиято стойност е адрес от паметта.

Пример:

```
int a=20, b=30,c;
```

```
int *pa, *pb; //дефинират ср указатели pa, pb към тип int.
```

ВАЖНО!!! Преди използването на указателите, да се инициализират (т.е. да им се дадат стойности)

```
pa=&a; //pa получава като стойност адреса на променливата a - казва се, че pa сочи към a
```

```
pb=&b; //pb получава като стойност адреса на променливата b - казва се, че pb сочи към b
```

След тази инициализация:

Навсякъде където се използва &a може да се използва pa.

Навсякъде където се използва &b може да се използва pb.

Навсякъде където се използва a може да се използва *pa.

Навсякъде където се използва b може да се използва *pb.

Ако адресът на a е 0xff00 (примерно) променливата pa ще получи стойност 0xff00.

Ако адресът на b е 0xff04 (примерно) променливата pb ще получи стойност 0xff04

Пример:

```
c=a;
```

```
a=b;
```

```
b=c;
```

е еквивалентно на:

```
c=*pa; //c получава стойността на променливата, към която сочи pa
```

```
*pa=*pb; //променливата, към която сочи pa, получава стойността на променливата, към която сочи pb
```

```
*pb=c; //променливата, към която сочи pb, получава стойността на променливата c
```

```
void change(int *pa, int *pb) //формалните параметри се дефинират като указатели
{
    int c;
```

```

c=*pa;    //с получава стойността на променливата, към която сочи pa
*pa=*pb;  //променливата, към която сочи pa, получава стойността на променливата,
към която сочи pb
*pb=c;    //променливата, към която сочи pb, получава стойността на променливата c
}
int main()
{
    x=12; y=10;
    change(&x,&y); //като фактически параметри се предават адресите на x и y
    printf("x=%d y=%d",x,y);
    return 0;
}

```

Извеждат се: x=10 y=12;

Т.е. функцията change(&x,&y) разменя стойностите на x и y.

Ако искаме функцията да размени стойностите на друга двойка променливи (например p и q) извикваме я с фактически параметри p и q:

```
change(&p,&q);
```

Състояние на оперативната памет (ОП):

Преди извикване на функцията change() x=12 y=10

По време на изпълнение на change() x=12 y=10

Заделя се допълнителна памет за формалните параметри (pa, pb) и стойностите на фактическите (&x, &y) се копират в съответните формални параметри. Ако например адресите на x и y са 0xff00 и 0xff04

pa=0xff00 pb=0xff04

т.е. pa сочи към x, pb сочи към y

Тогава:

с получава стойността на променливата, към която сочи pa (т.е. стойността на x) -> c=12

променливата, към която сочи pa (т.е. x), получава стойността на променливата, към която сочи pb (т.е. y)

x=10

променливата, към която сочи pb (т.е. y), получава стойността на променливата c. y=12

След приключване на изпълнението на функцията change() x=10 y=12

Паметта, заделена за формалните параметри

pa и pb и вътрешната променлива c се освобождава.

Остават x и y с разменени стойности.

Задача: Да се състави функция за решаване на линейното уравнение $ax+b=0$

```
int uravn(float, float, float*); //декларация (прототип) на функцията
int main()
{
    float a,b,x;
    printf("a=");scanf("%f",&a);
    printf("b=");scanf("%f",&b);
    switch(uravn(a,b,&x))
    {
        case 1:printf("Every x");break;
        case 2:printf("No solution");break;
        case 3:printf("x=%f",x);break;
    }
    return 0;
}

int uravn(float a, float b, float *px)
{
    if(a==0)
    {
        if(b==0)return 1;
        else return 2;
    }
    else
    {
        *px=-b/a;
        return 3;
    }
}
```

Рекурсия. Рекурсивни функции

1. Понятие за рекурсия

Рекурсивен обект - обект, който съдържа себе си.

Рекурсивно изображение - изображение, което съдържа себе си.

Рекурсивна дефиниция - дефиниция, при която дадено понятие се дефинира чрез себе си.

Да разгледаме функцията $n!$ (n факториел)

$n! = 1, n=0$

$1*2*3*...*n$, за $n>0$

Тази дефиниция е итеративна (на база на тази дефиниция създаваме итеративен алгоритъм (алгоритъм, съдържащ цикъл) за изчисляване на $n!$

```
int fact(int n)
{
    int P=1;
    for(int i=1;i<=n;i++)
        P=P*i;
    return P;
}
```

$n! = 1, n=0$

$n*(n-1)!, n>0$

Тази дефиниция е рекурсивна - $n!$ се дефинира чрез $(n-1)!$

Всяка рекурсивна дефиниция има поне един прост случай, който се задава явно. В случая: $n!=1$ за $n=0$.

Пример: Да изчислим $5!$ по дадената дефиниция:

$5!=5*4!$

$4!=4*3!$

$3!=3*2!$

$2!=2*1!$

$1!=1*0!$

$0!=1$ (прост случай, зададен явно - нарича се дъно на рекурсията)

Дотук е т.нар. прав ход на рекурсията.

Сега се връщаме обратно към $5!$ с т.нар. обратен ход на рекурсията:

$1!=1*1=1$

$2!=2*1=2$

$3!=3*2=6$

$4!=4*6=24$

$5!=5*24=120$

2. Рекурсивна функция - функция, която извиква себе си

Пример - рекурсивна функция за изчисляване на $n!$ (на база на рекурсивната дефиниция):

```
int factr(int n)
{
    if(n==0)return 1;
    else return n*factr(n-1);
}
```



```

int main()
{
    int n;
    scanf("%d",&n);
    printf("\n%d",fact(n));//извикване на итеративната функция
    printf("\n%d",factr(n));//извикване на рекурсивната функция
}

```

Ефективност на рекурсивните функции по отношение на бързодействие и използвана памет:

- Използват повече памет и са по-бавни в сравнение с итеративните функции

Пример: Какво се случва при извикването на factr(5) от главната функция main()->factr(5)->factr(4)->factr(3)->factr(2)->factr(1)->factr(0) (дъно на рекурсията)

дотук е т.нар. прав ход на рекурсията. При всяко извикване на factr() се заделя допълнителна памет за съответния формален параметър. Следователно - използва се повече памет. За всяко извикване на функцията е необходимо време. Следователно - повече време за изпълнение.

При обратния ход на рекурсията всяка функция връща съответната стойност в точката на извикването си и паметта, заделена за нейния формален параметър се освобождава.

Защо се използват рекурсивни функции, след като са по-бавни и изискват повече памет?

Защото в някои случаи рекурсивното решение е доста по-лесно и компактно в сравнение с итеративното.

Задача. Да се състави функция за изчисляване на n-тото число на Фибоначи:

а/с използване на цикъл

б/с рекурсия

Дефиниция на редицата на Фибоначи:

$Fib(n)=1, n=0, n=1$

$Fib(n)=Fib(n-1) + Fib(n-2)$

т.е. първите две числа на Фибоначи са 1, а всяко следващо е сумата от предходните две.

```

int fibr(int n)
{
    if(n<2)return 1;
    else return fibr(n-1)+fibr(n-2);
}

```

```

int fib(int n)

```

```

{
    int a=1,b=1,c=1;
    for(int i=2;i<=n;i++)
    {
        c=a+b; //текущото число е сумата от предходните две
        a=b;
        b=c;
    }
    return c;
}

```

```

int main()
{
    int n;
    do{
        printf("n=? (n>=0):");
        scanf("%d",&n);
    }while(n<0);
    printf("\nfib(%d)=%d,n,fib(n));
    printf("\nfibr(%d)=%d,n,fibr(n));
    return 0;
}

```

При по-големи стойности на n (напр. 45) се забелязва видимо забавяне на изпълнението на рекурсивната функция (итеративната връща резултата веднага, а рекурсивната - след секунди). Това се дължи на т.нар. нелинейна рекурсия (всяка функция извиква себе си два пъти: $\text{fibr}(n)$ $\text{fibr}(n-1)$ $\text{fibr}(n-2)$ и оттам големия брой извиквания на функцията.

Задача: С използването на външна променлива изведете колко пъти е извикана функцията $\text{fibr}(n)$?